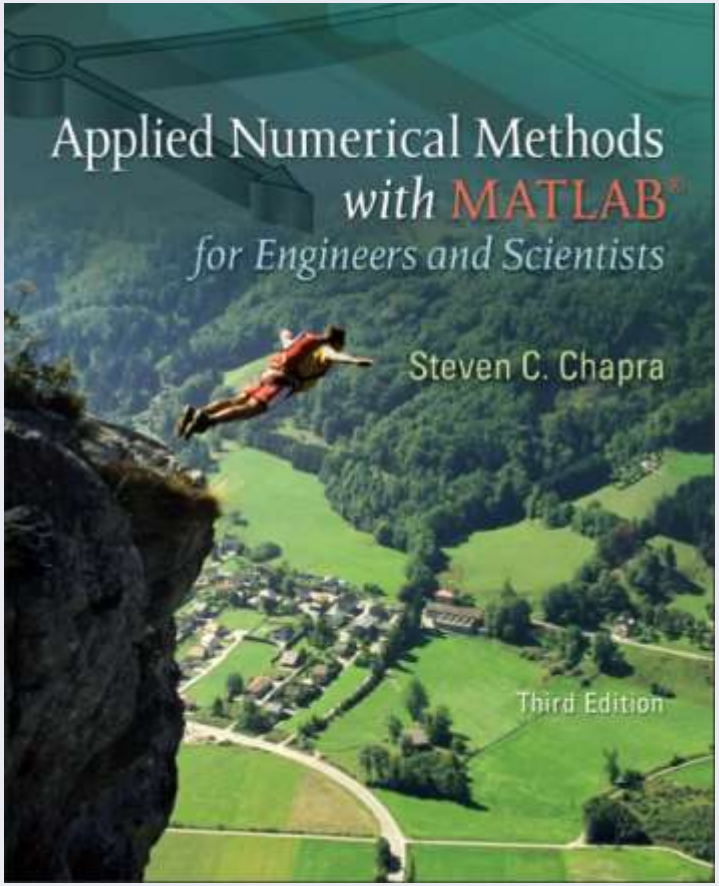


Part 1
Chapter 4

Roundoff and Truncation Errors

PowerPoints organized by Dr. Michael R. Gustafson II, Duke University and Prof. Steve Chapra, Tufts University

All images copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Chapter Objectives

- Understanding the distinction between **accuracy** and **precision**.
- Learning how to **quantify error**.
- Learning how error estimates can be used to decide **when to terminate** an iterative calculation.
- Understanding how **roundoff errors** occur because digital **computers have a limited ability** to represent numbers.
- Understanding why floating-point numbers have limits on their range and precision.

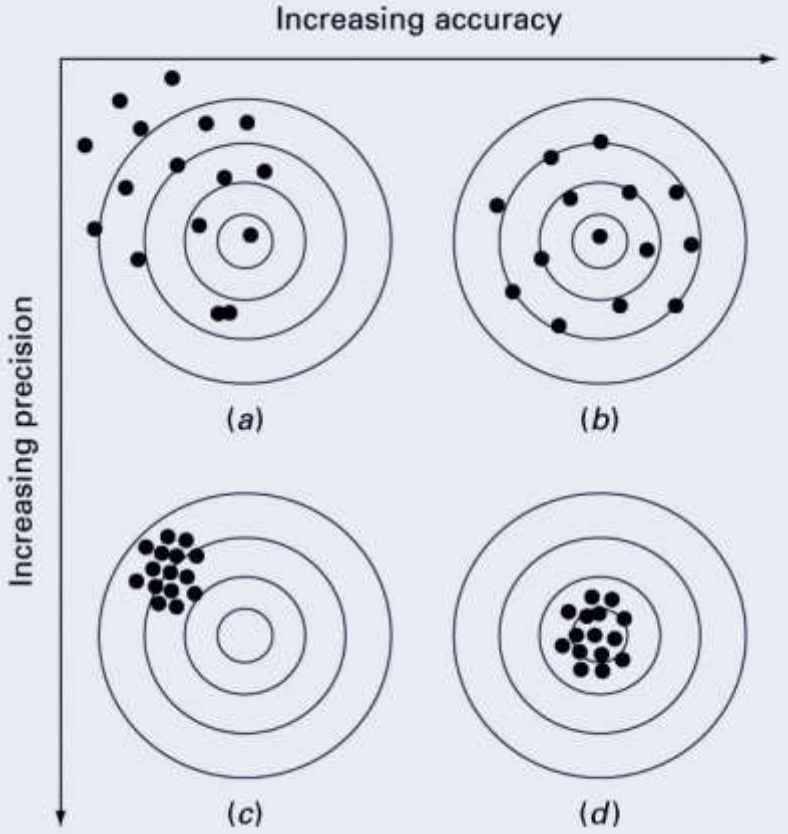
Objectives (cont)

- Recognizing that **truncation errors** occur when **exact mathematical formulations** are represented by approximations.
- Knowing how to **use the Taylor series to estimate truncation errors.**
- Understanding how to write forward, backward, and centered finite-difference **approximations of the first and second derivatives.**
- Recognizing that efforts to minimize truncation errors can sometimes increase roundoff errors.

Accuracy and Precision

- ***Accuracy*** refers to how closely a computed or measured value agrees with the true value, while *precision* refers to how **closely** individual computed or measured values **agree with each other**.

a) inaccurate and imprecise
b) accurate and imprecise
c) inaccurate and precise
d) accurate and precise



Error Definitions

- **True error (E_t):** the difference between the true value and the approximation.
- **Absolute error ($|E_t|$):** the absolute difference between the true value and the approximation.
- **True fractional relative error:** the true error divided by the true value.
- **Relative error (ε_t):** the true fractional relative error expressed **as a percentage.**

Error Definitions (cont)

- The previous definitions of error **relied on knowing a true value. If that is not the case, approximations** can be made to the error.
- **The approximate percent relative error** can be given as the **approximate error divided by the approximation**, expressed as a percentage - though this presents **the challenge of finding the approximate error!**
- **For iterative processes,** the error can be approximated as the difference in values between sucessive iterations.

Using Error Estimates

- Often, when performing calculations, we may **not be concerned with the sign of the error** but are interested in whether the absolute value of the percent relative error is lower than a prespecified **tolerance ε_s.** For such cases, the computation is repeated until

$$|\varepsilon_a| < \varepsilon_s$$

- This relationship is referred to as a ***stopping criterion*.**

Roundoff Errors

- *Roundoff errors* arise **because digital computers cannot represent some quantities exactly.** There are two major facets of roundoff errors involved in numerical calculations:
  - Digital computers have **size** and **precision limits** on their ability to represent numbers.
  - Certain numerical manipulations are highly sensitive to roundoff errors.

Computer Number Representation

- By default, MATLAB has adopted the **IEEE double-precision format** in which eight bytes (64 bits) are used to represent floating-point numbers:
  $n = \pm(1+f) \times 2^e$
- The sign is determined by a sign bit
- The mantissa *f* is determined by a 52-bit binary number
- The exponent *e* is determined by an 11-bit binary number, from which 1023 is subtracted to get *e*

Floating Point Ranges

- Values of –1023 and +1024 for *e* are reserved for special meanings, so the exponent range is –1022 to 1023.
- The largest possible number MATLAB can store has
  - *f* of all 1's, giving a significand of $2 - 2^{-52}$, or approximately 2
  - *e* of 1111111110_2, giving an exponent of 2046 – 1023 = 1023
  - This yields approximately $2^{1024} = 1.7997 \times 10^{308}$
- The smallest possible number MATLAB can store with full precision has
  - *f* of all 0's, giving a significand of 1
  - *e* of 0000000001_2, giving an exponent of 1–1023 = –1022
  - This yields $2^{-1022} = 2.2251 \times 10^{-308}$

Floating Point Precision

- The 52 bits for the mantissa *f* correspond to about 15 to 16 base-10 digits. The machine epsilon - the maximum relative error between a number and MATLAB's representation of that number, is thus
  $2^{-52} = 2.2204 \times 10^{-16}$

Roundoff Errors with Arithmetic Manipulations

- Roundoff error can happen in several circumstances other than just storing numbers - for example:
  - *Large computations* - if a process performs a large number of computations, roundoff errors may build up to become significant
  - *Adding a Large and a Small Number* - Since the small number's mantissa is shifted to the right to be the same scale as the large number, digits are lost
  - *Smearing* - Smearing occurs whenever the individual terms in a summation are larger than the summation itself.
    - $(x + 10^{-20}) - x = 10^{-20}$ mathematically, but
      x = 1; $(x + 10^{-20}) - x$ gives a 0 in MATLAB!

Truncation Errors

- *Truncation errors* are those that result from using an approximation in place of an exact mathematical procedure.
- Example 1: approximation to a derivative using a finite-difference equation:

$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$$

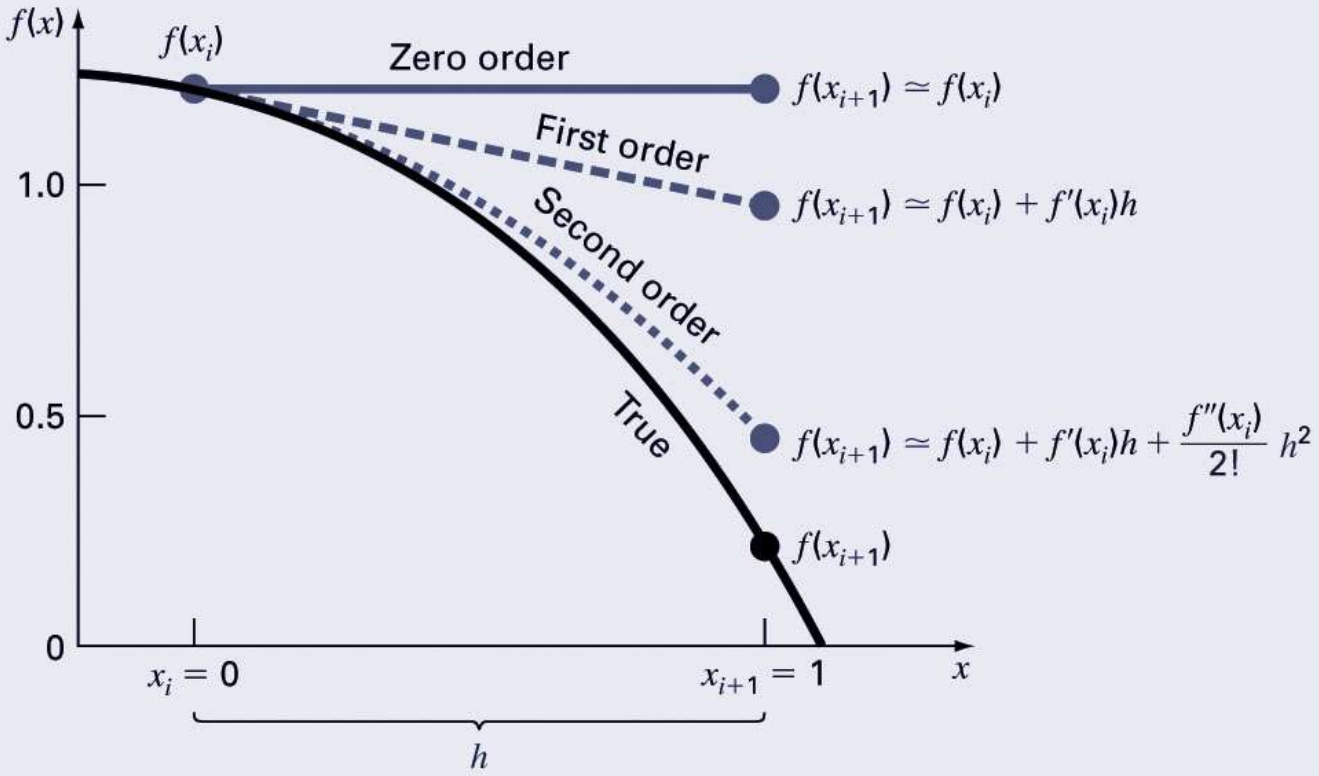
- Example 2: The Taylor Series

The Taylor Theorem and Series

- The *Taylor theorem* states that any smooth function can be approximated as a polynomial.
- The *Taylor series* provides a means to express this idea mathematically.

The Taylor Series

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \frac{f^{(3)}(x_i)}{3!}h^3 + \cdots + \frac{f^{(n)}(x_i)}{n!}h^n + R_n$$



Truncation Error

- In general, **the *n*th order Taylor series expansion will be exact for an *n*th order polynomial.**
- In other cases, the remainder term R_n is of the order of h^{n+1}, meaning:
  - The more terms are used, the smaller the error, and
  - The smaller the spacing, the smaller the error for a given number of terms.

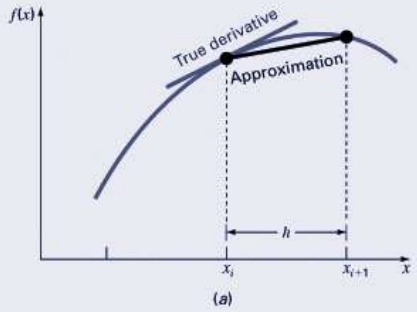
Numerical Differentiation

- The first order Taylor series can be used to calculate approximations to derivatives:
  - Given: $f(x_{i+1}) = f(x_i) + f'(x_i)h + O(h^2)$
  - Then: $f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} + O(h)$
- This is termed a “forward” difference because it utilizes data at *i* and *i*+1 to estimate the derivative.

Differentiation (cont)

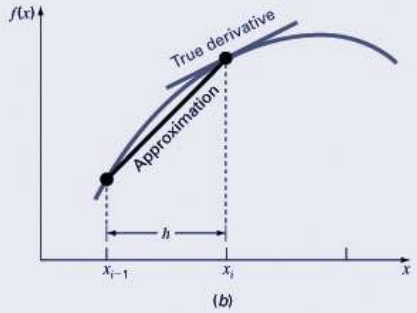
- There are also backward and centered difference approximations, depending on the points used:
- Forward:

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} + O(h)$$



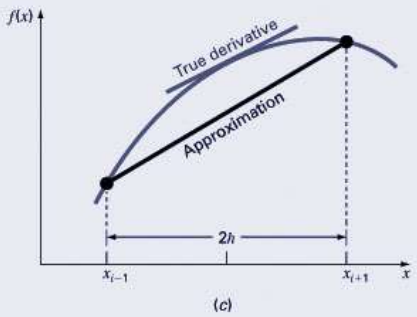
- Backward:

$$f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{h} + O(h)$$



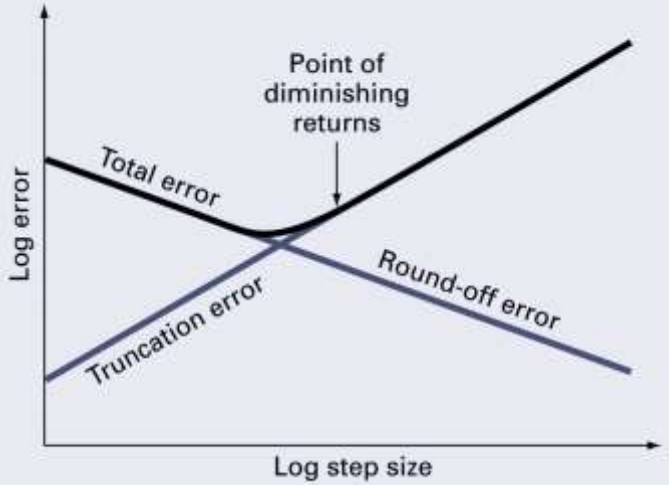
- Centered:

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{2h} + O(h^2)$$



Total Numerical Error

- The *total numerical error* is the summation of the truncation and roundoff errors.
- The truncation error generally *increases* as the step size increases, while the roundoff error *decreases* as the step size increases - this leads to a point of diminishing returns for step size.

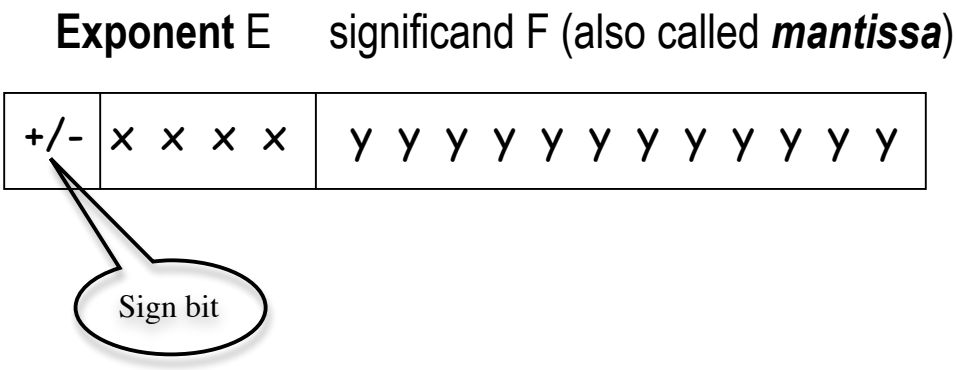


Other Errors

- Blunders - errors caused by malfunctions of the computer or human imperfection.
- Model errors - errors resulting from incomplete mathematical models.
- Data uncertainty - errors resulting from the accuracy and/or precision of the data.

Floating point Representation of Numbers

FP is useful for representing a number in a wide range: **very small** to **very large**. It is widely used in the scientific world. Consider, the following FP representation of a number

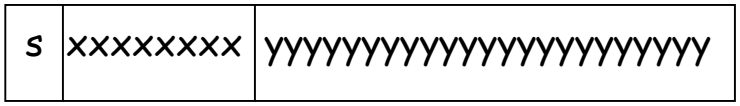


In **decimal** it means (+/-) **1**. yyyyyyyyyyy x 10^{xxxx}

In **binary**, it means (+/-) **1**. yyyyyyyyyyy x 2^{xxxx}

(The 1 is implied)

IEEE 754 single-precision (32 bits)



Single precision

1 8 23 bits

Largest = 1. 1 1 1 … x 2^{+127} ≈ 2 x 10^{+38}

Smallest = 1.000 … x 2^{-128} ≈ 1 x 10^{-38}

These can be positive and negative, depending on s.

(But there are exceptions too)

IEEE 754 double precision (64 bits)

S	exponent	significand

1 11 bits 52 bits

Largest = 1. 1 1 1… x 2^{+1023}

Smallest = 1.000… X 2^{-1024}

Overflow and **underflow** in FP

An **overflow** occurs when the number if too large to fit in the frame. An **underflow** occurs when the number is too small to fit in the given frame.

How do we represent zero?

IEEE standards committee solved this by making **zero** a special case: if every bit is zero (the sign bit being irrelevant), then the number is considered zero.

Then how do we represent 1.0?

It should have been 1.0×2^0 (same as 0)! The way out of this is that the interpretation of the exponent bits is not straightforward. The exponent of a single-precision float is **"shift-127" encoded** (**biased representation**), meaning that the actual exponent is (xxxxxxx minus 127). So thankfully, we can get an exponent of zero by storing 127.

Exponent = 11111111 (i.e. 255) means 255-127 = 128
Exponent = 01111111 (i.e. 127) means 127-127 = 0
Exponent = 00000001 (i.e. 1) means 1-127 = -126

The consequence of shift-127

Exponent = 00000000 (reserved for 0) can no more be used to represent the smallest number. We forego something at the low extreme of the spectrum of representable magnitudes, which should be 2^{-127}

It seems wise, to give up the smallest exponent instead of giving up the ability to represent 1 or zero!

More special cases

Zero is not the only "special case" float. There are also representations for positive and negative infinity, and for a not-a-number (NaN) value, for results that do not make sense (for example, non-real numbers, or the result of an operation like infinity times zero). How do these work? A number is infinite if every bit of the exponent is 1 (yes, we lose another one), and is NaN if every bit of the exponent is 1 plus any mantissa bits are 1. The sign bit still distinguishes +/-inf and +/-NaN. Here are a few sample floating point representations:

Exponent	Mantissa	Object
0	0	Zero
0	Nonzero	Denormalized number*
1-254	Anything	+/- FP number
255	0	+ / - infinity
255	Nonzero	NaN

* Any non-zero number that is smaller than the smallest normal number is a denormalized number. The production of a denormal is sometimes called gradual underflow because it allows a calculation to lose precision slowly when the result is small.

Floating Point Addition

Example using decimal

$A = 9.999 \times 10^{1}$, $B = 1.610 \times 10^{-1}$, $A+B = ?$

Step 1. Align the smaller exponent with the larger one.

$B = 0.0161 \times 10^{1} = 0.016 \times 10^{1}$ (round off)

Step 2. Add significands

$9.999 + 0.016 = 10.015$, so $A+B = 10.015 \times 10^{1}$

Step 3. Normalize

$A+B = 1.0015 \times 10^{2}$

Step 4. Round off

$A+B = 1.002 \times 10^{2}$

Now, try to add 0.5 and –0.4375 in binary.

Floating Point Multiplication

Example using decimal

$A = 1.110 \times 10^{10}$, $B = 9.200 \times 10^{-5}$ $A \times B = ?$

Step 1. Exponent of $A \times B = 10 + (-5) = 5$

Step 2. Multiply significands

$1.110 \times 9.200 = 10.212000$

Step 3. Normalize the product

$10.212 \times 10^{5} = 1.0212 \times 10^{6}$

Step 4. Round off

$A \times B = 1.021 \times 10^{6}$

Step 5. Decide the sign of $A \times B$ $(+ \times + = +)$

So, $A \times B = + 1.021 \times 10^{6}$

Organizing and Debugging MATLAB Programs

Gerald Recktenwald
Portland State University
Department of Mechanical Engineering

These slides are a supplement to the book *Numerical Methods with* MATLAB*: Implementations and Applications*, by Gerald W. Recktenwald, © 2000, Prentice-Hall, Upper Saddle River, NJ. These slides are © 2000 Gerald W. Recktenwald. The PDF version of these slides may be downloaded or stored or printed only for noncommercial, educational use. The repackaging or sale of these slides in any form, without written consent of the author, is prohibited.

The latest version of this PDF file, along with other supplemental material for the book, can be found at `www.prenhall.com/recktenwald`.

Version 0.9 October 10, 2000

Overview

- Rationale
- Programming Style
- Why and How of Modular Code
- Top down program design
- Basic Debugging

Rationale

Organized programs are. . .

- easier to maintain
- easier to debug
- not much harder to write

Debugging. . .

- is inevitable
- can be anticipated with good program design
- can be done interactively with MATLAB 5.x

Programming Style (1)

A consistent programming style gives your programs a visual familiarity that helps the reader quickly comprehend the intention of the code.

A programming style consists of

- Visual appearance of the code
- Conventions used for variable names
- Documentation with comment statements

Programming Style (2)

Use visual layout to suggest organization

- Indent `if...end` and `for...end` blocks
- Blank lines separate major blocks of code

Example: *Indent code for conditional structures and loops*

```
if condition 1 is true

    Block 1

elseif condition 2 is true

    Block 2

end



for i=1:length(x)

    Body of loop

end
```

Programming Style (3)

Use meaningful variable names

```
d = 5;
t = 0.02;
r = d/2;
r2 = r + t;
```

```
d_in = 5;
thick = 0.02;
r_in = d_in/2;
r_out = r_in + thick;
```

Follow Programming and Mathematical Conventions

Variable names	Typical usage
`i`, `j`, `k`	Array subscripts, loop counters
`i`, `j`	$\sqrt{-1}$ with complex arithmetic
`m`, `n`	End of a sequence, $i = 1, \ldots, n$, number of rows (m) and columns (n) in a matrix
`A`, `B`	generic matrix
`x`, `y`, `z`	generic vectors

Note: Consistency is more important than convention.

Programming Style (4)

Note: I prefer to avoid use of lower case "L" as a variable name. It looks a lot like the number "1". Which of the following statements assigns the value "1" to the lower case version of the variable "L"?

```
l = 1;          (or)          1 = l;
```

Programming Style (5)

Document code with comment statements

- Write comments as you write code, not after
- Include a prologue that supports "help"
- Assume that the code is going to be used more than once
- Comments should be short notes that augment the meaning of the program statements: Do not parrot the code.
- Comments alone do not create good code.
  - ▷ You cannot fix a bug by changing the comments

Programming Style (6)

Example: *Comments at beginning of a block*

```
% --- Evaluate curve fit and plot it along with original data
tfit = linspace(min(t),max(t));
pfit = polyval(c,tfit);
plot(t,p,'o',tfit,pfit,'--');
xlabel('Temperature (C)');   ylabel('Pressure  (MPa)');
legend('Data','Polynomial Curve Fit');
```

Example: *Short comments at side of statements*

```
cp = 2050;    %  specific heat of solid and liquid paraffin   (J/kg/K)
rho = 810;    %  density of liquid or solid paraffin  (kg/m^3)
k = 0.23;     %  thermal conductivity,  (W/m/C)
L = 251e3;    %  latent heat  (J/kg)
Tm = 65.4;    %  melting temperature  (C)
```

Supporting On-line Help

- First line of a function is the definition
- Second line must be a comment statement
- All text from the second line up to the first non-comment is printed in response to

  `help` *`functionName`*

Prologue Used in the NMM Toolbox

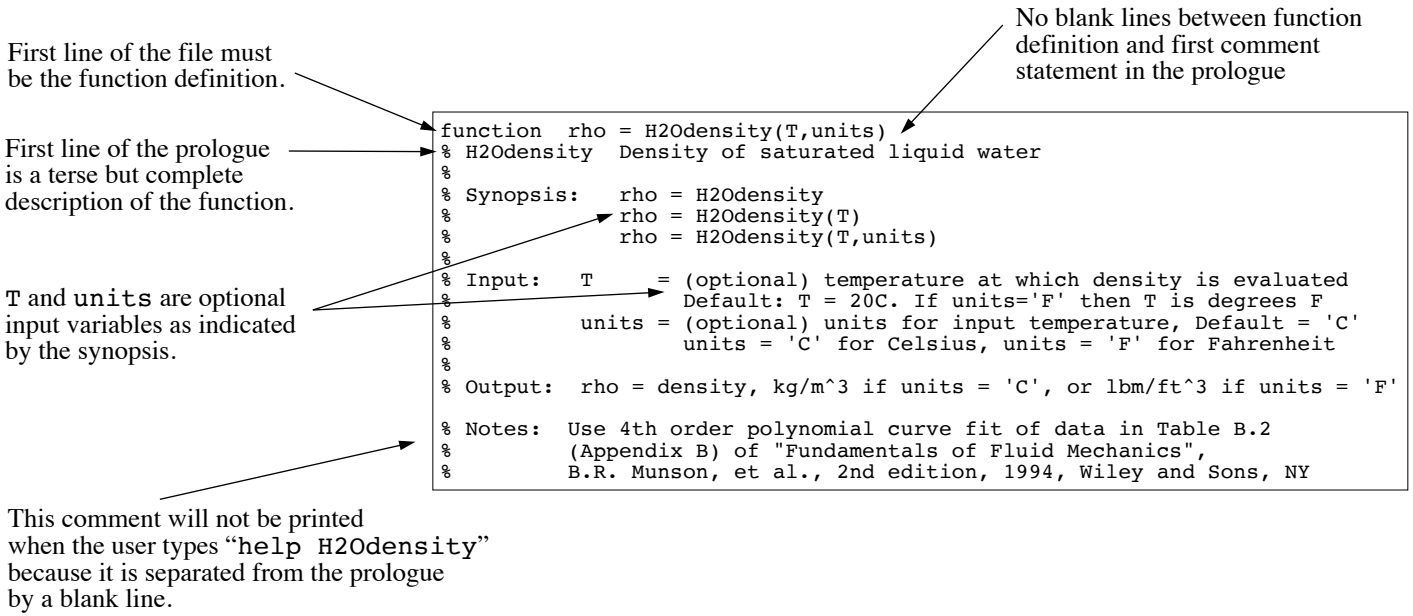
Summary: One line description of what the function does.

Synopsis: Lists the various ways in which the function can be called.

Input: Describes each input variable.

Output: Describes each output variable.

Function Prologue



Modular Code (1)

A module should be dedicated to *one* task

- Flexibility is provided by input/output parameters

General purpose modules need. . .

- Description of input/output parameters
- Meaningful error messages so that user understands the problem

Modular Code (2)

Reuse modules

- Debug once, use again
- Minimize duplication of code
- Any improvements are available to all programs using that module
- Error messages must be meaningful so that user of general purpose routine understands the problem

Organization takes experience

- Goal is *not* to maximize the number of m-files
- Organization will evolve on complex projects

Example: Built-in Bessel functions (1)

The Bessel functions are solutions to

$$z^2\frac{d^2y}{dz^2} + z\frac{dy}{dz} - (z^2 + \nu^2)y = 0$$

The Bessel function of the first kind is

$$J_\nu(z) = \left(\frac{z}{2}\right)^\nu \sum_{k=0}^{\infty} \frac{\left(\frac{z^2}{4}\right)^k}{k!\,\Gamma(\nu + k + 1)}$$

where ν is a real number, z is complex, $i = \sqrt{-1}$ and

$$\Gamma(z) = \int_0^\infty e^{-t}t^{z-1}\,dt$$

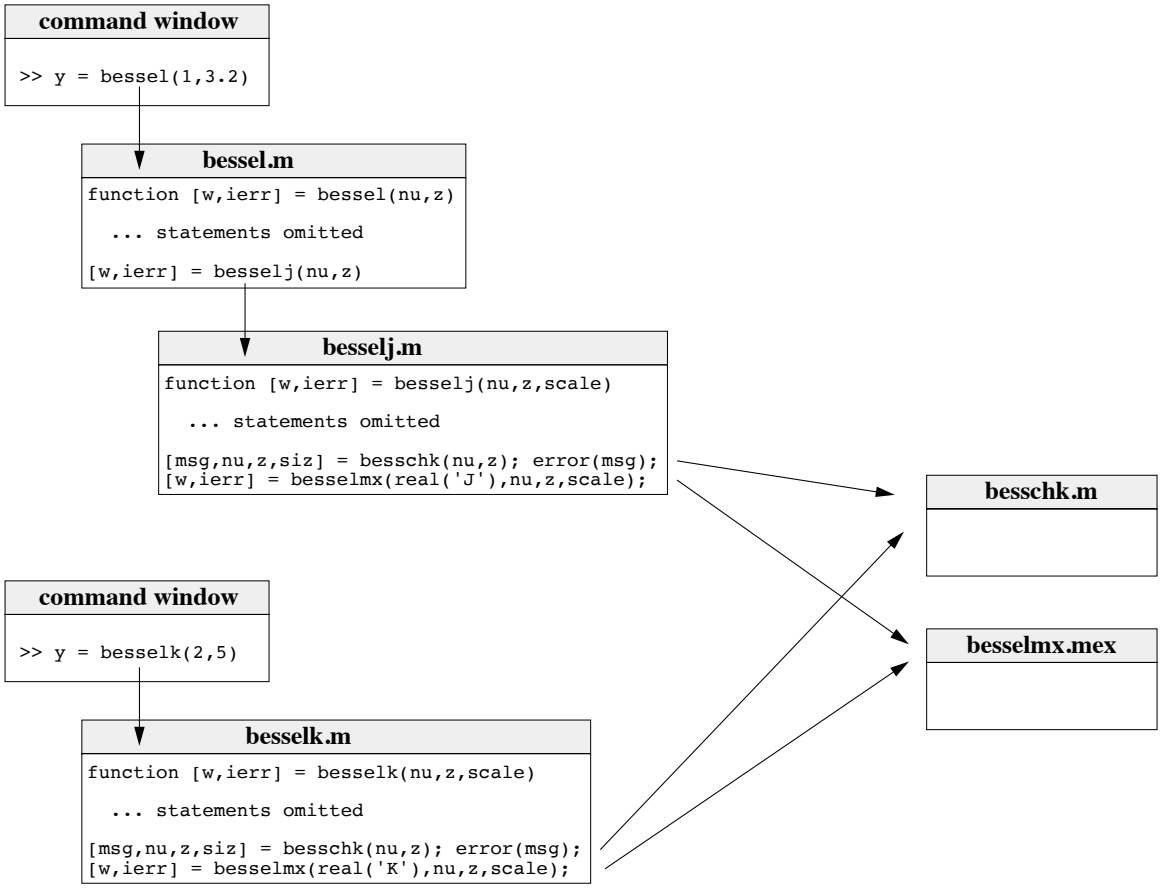
Other Bessel functions (which are also solutions to the ODE) are defined in terms of $J_\nu(z)$.

Example: Built-in Bessel functions (2)

Rather than repeat the code that computes $J_\nu(z)$ and $\Gamma(z)$, these fundamental functions are part of a core routine that gets evaluated via an interface function.

```
>> lookfor bessel
BESSCHK  Check arguments to bessel functions.
BESSEL   Bessel functions of various kinds.
BESSELA  Obsolete Bessel function.
BESSELH  Bessel function of the third kind (Hankel function).
BESSELI  Modified Bessel function of the first kind.
BESSELJ  Bessel function of the first kind.
BESSELK  Modified Bessel function of the second kind.
BESSELY  Bessel function of the second kind.
BESSLDEM Driver function for Bessel zero finding.
BESSLODE Bessel's equation of order 0 used by BESSLDEM.
```

Example: Built-in Bessel functions (3)



Defensive Programming

- Do not assume the input is correct. Check it.
- Provide a "catch" or default condition for a `if...elseif...else...` construct
- Include optional (verbose) print statements that can be switched on when trouble occurs
- Provide diagnostic error messages.

Example: H2Odensity.m

```
 1  function  rho = H2Odensity(T,units)
 2  % H2Odensity  Density of saturated liquid water
 3  %
 4  % Synopsis:   rho = H2Odensity
 5  %             rho = H2Odensity(T)
 6  %             rho = H2Odensity(T,units)
 7  %
 8  % Input:   T     = (optional) temperature at which density is evaluated
 9  %                  Default: T = 20C. If units='F', then T is degrees F
10  %          units = (optional) units for input temperature, Default = 'C'
11  %                  units = 'C' for Celsius, units = 'F' for Fahrenheit
12  %
13  % Output:  rho = density, kg/m^3 if units = 'C', or lbm/ft^3 if units = 'F'
14
15  % Notes:  Use 4th order polynomial curve fit of data in Table B.2
16  %         (Appendix B) of "Fundamentals of Fluid Mechanics",
17  %         B. R. Munson, et al., 2nd edition, 1994, Wiley and Sons, NY
18
19  if nargin<1
20    rho = 998.2;  return;    %  Density at 20 C w/out evaluating curve fit
21  elseif nargin==1
22    units='C';               %  Default units are C
23  end
24
25  % --- Convert to degrees C if necessary
26  if upper(units)=='F'
27    Tin = (T-32)*5/9;        %  Convert F to C; don't change input variable
28  elseif upper(units) == 'C'
29    Tin = T;
30  else
31    error(sprintf('units = ''%s'' not allowed in H2Odensity',units));
32  end
33
34  % --- Make sure temperature is within range of curve fit
```

```
35  if Tin<0 | Tin>100
36    error(sprintf('T = %f (C) is out of range for density curve fits',Tin));
37  end
38
39  % --- Curve fit coefficients
40  c = [ 1.543908249780381441e-05  -5.878005395030049852e-03 ...
41        1.788447211945859774e-02  1.000009926781338436e+03];
42
43  rho = polyval(c,Tin);     % Evaluate polynomial curve fit
44  if upper(units)=='F'
45    rho = rho*6.243e-2;     % Convert kg/m^3 to lbm/ft^3
46  end
```

Preemptive Debugging

- Use defensive programming

- Break large programming projects into modules
  - ▷ Develop reusable tests for key modules
  - ▷ Good test problems have known answers
  - ▷ Run the tests after changes are made to the module

- Include diagnostic calculations in a module
  - ▷ Enclose diagnostics inside `if...end` blocks so that they can be turned off.
  - ▷ Provide extra print statements that can also be turned on and off

Debugging Tools

- MATLAB version 5 (and later) has an *interactive* debugger
- The `type` and `dbtype` commands are used to list contents of an m-file.
- The `error` function prints a message to the screen, and stops execution. This provides for graceful failure, and the opportunity to inform the reader of potential causes for the error.
- The `warning` function prints a message to the screen, but does not stop execution.
- `pause` or `keyboard` commands can be used to temporarily halt execution.

Use of `keyboard` command

```
function r = quadroot(a,b,c)
% quadroot   Roots of quadratic equation and demo of keyboard command
%
% Synopsis:  r = quadroot(a,b,c)
%
% Input:     a,b,c = coefficients of  a*x^2 + b*x + c = 0
%
% Output:    r = column vector containing the real or complex roots

% See Chapter 4, Unavoidable Errors in Computing, for a discussion
% of the formula for r(1) and r(2)
d = b^2 - 4*a*c;
if d<0
  fprintf('Warning in function QUADROOT:\n');
  fprintf('\tNegative discriminant\n\tType "return" to continue\n');
  keyboard;
end
q = -0.5*( b + sign(b)*sqrt(b^2 - 4*a*c) );

r = [q/a; c/q];  % store roots in a column vector
```